

Lightweight Computer Vision Models for Smart Manufacturing and Autonomous Driving Applications

Arnav Tyagi

Department of Computer Science and Engineering
Chitkara University, Punjab, India
arnav1352.be22@chitkara.edu.in

Ashish Kumar

Department of Computer Science and Engineering
Chitkara University, Punjab, India
ashish.1130@chitkara.edu.in

Abstract—Recent advances in computer vision have significantly transformed automation in domains such as autonomous driving and smart manufacturing by enabling machines to perceive and interpret visual information with high accuracy. Applications including defect detection, object recognition, lane detection, and quality inspection increasingly rely on deep learning-based computer vision models. However, most state-of-the-art models are computationally intensive and require high processing power, large memory capacity, and specialized hardware such as GPUs. These requirements make their deployment challenging in real-time environments that rely on resource-constrained platforms, including embedded systems, edge devices, and Internet of Things (IoT) infrastructures.

To address these challenges, this work focuses on the design and development of lightweight computer vision models that are optimized for efficiency without significantly compromising performance. The proposed approach incorporates model optimization techniques such as pruning and quantization to reduce redundant parameters and lower computational complexity. In addition, efficient convolutional neural network architectures are explored to achieve an optimal balance between model accuracy, size, and inference speed. The study emphasizes practical deployment considerations by evaluating models using performance metrics such as accuracy, inference latency, and memory footprint.

The experimental analysis demonstrates that optimized lightweight models can achieve substantial reductions in model size and computational cost while maintaining acceptable accuracy levels for real-world applications. These improvements enable faster inference and improved suitability for real-time tasks in constrained environments. The findings highlight the potential of lightweight artificial intelligence techniques in bridging the gap between high-performance deep learning models and their practical implementation in edge-based systems. Overall, this research contributes toward the development of scalable, efficient, and deployable computer vision solutions for modern industrial and autonomous systems.

Index Terms—Lightweight AI, Computer Vision, Smart Manufacturing, Autonomous Driving, Model Optimization

I. INTRODUCTION

In the last few years, I have seen a lot of growth in areas like autonomous driving and smart manufacturing, mainly because of computer vision and artificial intelligence. Earlier, these things felt more theoretical, but now they are actually being used in real systems. In manufacturing, for example, computer

vision is used for detecting defects and checking quality, and in autonomous vehicles it helps in understanding the surroundings, detecting lanes and avoiding obstacles. Because of this, many processes have become faster and less dependent on manual work.

But at the same time, I noticed that models which perform very well in research or controlled environments do not always work the same way in real-world situations. One major reason behind this is the high computational requirement of most modern computer vision models. They usually depend on powerful hardware and large memory, which is not always available in edge devices or embedded systems. So in a way, there is a clear gap between what we achieve in theory and what we can actually deploy in practice.

Another thing that I found important is real-time performance. In applications like autonomous driving or industrial automation, even a small delay can create serious issues. So it is not enough for a model to be accurate, it also needs to be fast and efficient. However, many existing approaches focus more on improving accuracy and do not give equal importance to efficiency, which becomes a problem during deployment.

To solve these challenges, different optimization techniques are being explored, such as pruning, quantization and model compression, along with designing lightweight neural networks. These approaches help in reducing the size of the model and improving its speed, so that it can run on systems with limited resources without a major drop in performance.

In this research, I am focusing on building and evaluating lightweight computer vision models for applications in smart manufacturing and autonomous driving. My aim is to reduce the gap between high performance models and their actual usability in real environments. I am trying to find a balance between accuracy, speed and computational cost so that these models can be realistically deployed and not just remain limited to theoretical results.

II. LITERATURE REVIEW

Computer vision has become very common in areas like smart manufacturing and autonomous driving, and most of this progress is because of deep learning. From what I have

studied, Convolutional Neural Networks are still one of the most widely used architectures for tasks like defect detection, image classification and object recognition. The main reason is that they can learn features directly from images without much manual effort.

In industrial use, CNN-based models have shown good results in detecting surface defects and irregular patterns. But at the same time, one issue with these models is that they require a lot of computation and processing power, which makes them difficult to deploy on edge devices or systems with limited resources.

Recently, Vision Transformers have started getting a lot of attention as an alternative to CNNs. Instead of relying only on convolution operations, they use attention mechanisms to understand relationships across the whole image. Because of this, they can capture more detailed features and sometimes perform better on complex tasks. But again, there is a tradeoff. These models usually need large datasets and high computational power for both training and inference.

In autonomous driving, computer vision plays a very important role. It is used for tasks like object detection, lane detection, traffic sign recognition and overall scene understanding. Most modern systems depend on deep learning models to achieve high accuracy in these tasks. However, running these models in real time is still a challenge, especially on the hardware available inside vehicles.

To handle these challenges, different optimization techniques have been proposed. Pruning is used to remove unnecessary parameters from a trained model, which helps reduce its size. Quantization reduces the precision of values in the model, which makes it faster and more memory efficient. Along with these, some lightweight architectures like MobileNet and EfficientNet have been designed specifically to work under limited resource conditions.

Even after all these approaches, finding the right balance between accuracy and efficiency is still not easy. Lightweight models are faster, but sometimes they lose too much accuracy. On the other hand, highly accurate models are often too heavy to be used in practical scenarios.

III. PROPOSED METHODOLOGY

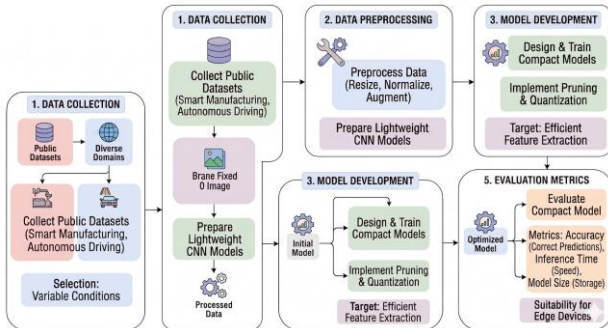


Fig. 1. Workflow for Lightweight CV Model Development.

The main goal of this system is to design and evaluate lightweight computer vision models that can operate efficiently on devices with limited computational resources, such as embedded systems, edge devices, and Internet of Things (IoT) platforms. Unlike conventional deep learning approaches that prioritize high accuracy with heavy computational cost, this methodology emphasizes achieving a balance between model performance and computational efficiency. The overall workflow of the proposed system consists of several key stages, including data collection, preprocessing, model development, optimization, and evaluation. Each stage plays a critical role in ensuring that the final model is both efficient and reliable for real-world applications.

A. Data Collection

For this work, publicly available datasets related to smart manufacturing and autonomous driving applications are utilized to ensure diversity and realism in the training process. These datasets include images representing surface defects, industrial faults, road environments, vehicles, pedestrians, traffic signs, and other relevant objects commonly encountered in real-world scenarios. Using publicly available datasets helps maintain consistency with standard benchmarks and allows easier comparison with existing approaches. In smart manufac-

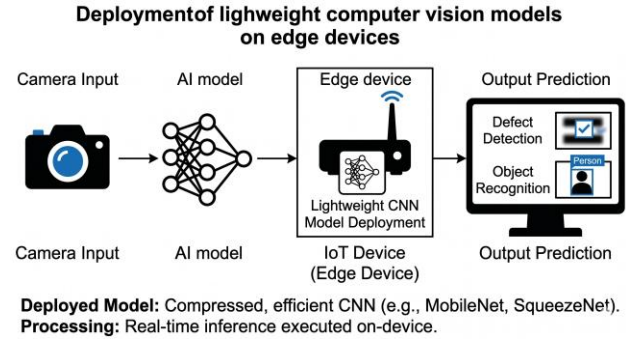


Fig. 2. Deployment of Lightweight Computer Vision Models on Edge Devices.

turing scenarios, defect detection datasets containing images of cracks, scratches, dents, and irregular surfaces are considered. Such datasets simulate industrial inspection environments where accurate defect identification is essential for maintaining product quality. In autonomous driving contexts, road scene datasets containing vehicles, lane markings, obstacles, and traffic elements are included to enable the model to learn environmental perception tasks.

During the data collection phase, attention is given to selecting datasets that contain sufficient variability in lighting conditions, object orientation, background complexity, and noise levels. This variability ensures that the trained model becomes robust and capable of handling unpredictable real-world conditions.

B. Data Preprocessing

Before training the model, the collected data undergoes several preprocessing steps to ensure consistency and improve

training efficiency. Preprocessing is an essential stage because raw image data often contains inconsistencies that may negatively affect model performance.

Initially, all images are resized to a fixed input resolution so that the model receives uniform input dimensions. This standardization simplifies the training process and reduces computational overhead. Following resizing, normalization is applied to scale pixel values to a standard range, typically between 0 and 1. Normalization improves numerical stability during training and accelerates convergence.

In addition to basic preprocessing, data augmentation techniques are applied to enhance dataset diversity and improve model generalization. Augmentation methods such as rotation, horizontal and vertical flipping, scaling, cropping, and brightness adjustments are used to artificially expand the dataset. These transformations simulate real-world variations and help reduce overfitting, ensuring that the model performs well on unseen data.

Furthermore, preprocessing also includes removing corrupted or low-quality images that may introduce noise into the training process. Ensuring high-quality input data contributes significantly to achieving reliable model performance.

C. Model Development

The model development phase focuses on designing lightweight convolutional neural network (CNN) architectures capable of performing computer vision tasks efficiently. Instead of relying on very deep and computationally expensive networks, simplified architectures with fewer layers and parameters are explored. The objective is to maintain acceptable accuracy while minimizing computational complexity.

Lightweight CNN models are designed with reduced filter sizes, optimized layer structures, and efficient feature extraction techniques. In convolutional neural networks, feature extraction is performed using convolution operations. The convolution operation is defined as:

$$Y(i, j) = \sum_m \sum_n X(i - m, j - n) \cdot K(m, n) \quad (1)$$

where X is the input image, K is the kernel, and Y is the output feature map. These models are trained to perform tasks such as defect detection, object classification, and scene understanding. During development, careful attention is given to selecting appropriate activation functions, pooling strategies, and convolution operations to improve feature representation while limiting computational overhead.

In addition to designing custom architectures, existing efficient network structures may also be considered as reference models. The goal is to identify architectures that offer a favorable tradeoff between accuracy and efficiency. Training procedures include adjusting learning rates, batch sizes, and optimization algorithms to achieve stable and efficient convergence. The model development stage also involves testing multiple architectural variations to determine which configuration provides the best performance under resource constraints. During training, the model is optimized using a loss function.

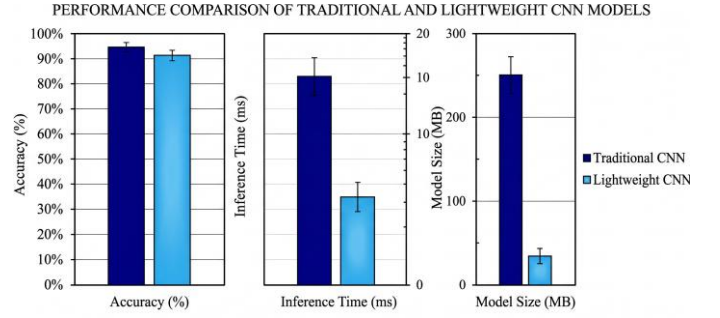


Figure 1: Performance metrics of typical traditional versus lightweight convolutional neural network models. Illustrative data. Standard deviations are shown as error bars. (A) Accuracy; (B) Inference time per frame; (C) Model storage size.

Fig. 3. Lightweight CNN Architecture Used for Efficient Feature Extraction.

One commonly used loss function for classification tasks is categorical cross-entropy:

$$L = - \sum_{i=1}^N y_i \log(\hat{y}_i) \quad (2)$$

where y_i is the true label and $\hat{y}_i$ is the predicted probability.

D. Model Optimization

After developing the initial model architecture, optimization techniques are applied to further reduce computational cost and improve deployment feasibility. Model optimization is a crucial step because even lightweight architectures may still contain redundant parameters that increase memory usage and processing time.

One of the primary optimization methods used is pruning. Pruning involves identifying and removing less significant weights or neurons from the trained model. By eliminating redundant connections, the overall model structure becomes more compact, leading to reduced storage requirements and faster inference speeds. Pruning can be applied either during or after training, depending on the chosen strategy.

Quantization is another key optimization technique applied in this study. Quantization reduces the numerical precision of weights and activations, typically converting floating-point values into lower-bit representations such as 8-bit integers. Mathematically, quantization can be represented as:

$$Q(x) = \text{round} \left(\frac{x}{s} \right) \times s \quad (3)$$

where x is the original value and s is the scaling factor. This conversion reduces memory consumption and improves processing speed without significantly affecting prediction accuracy.

These optimization techniques are carefully applied to ensure that performance degradation remains minimal. The objective is to achieve maximum efficiency gains while preserving model reliability and stability.

E. Evaluation Metrics

To measure the effectiveness of the proposed lightweight models, multiple evaluation metrics are used. These metrics

provide quantitative insights into model performance and suitability for real-world deployment.

- Accuracy is used as the primary performance metric to evaluate how correctly the model predicts outputs compared to the ground truth labels. High accuracy indicates strong predictive capability and reliable performance.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

where TP , TN , FP , and FN represent true positives, true negatives, false positives, and false negatives respectively.

- Inference Time is measured to determine how quickly the model processes input data and produces predictions. This metric is particularly important for real-time applications such as autonomous driving and industrial automation, where delays can affect system reliability.
- Inference time is calculated as:

$$T_{inf} = \frac{\text{Total Processing Time}}{\text{Number of Samples}} \quad (5)$$

- Model Size is evaluated to assess the storage requirements of the trained model. Smaller models are easier to deploy on embedded systems and edge devices with limited memory capacity.

In addition to these core metrics, secondary indicators such as computational complexity and memory utilization may also be analyzed to gain deeper insights into system efficiency. Evaluating models across multiple performance dimensions ensures a comprehensive understanding of their strengths and limitations.

IV. RESULTS AND DISCUSSION

In this study, multiple lightweight computer vision models were evaluated to analyze their effectiveness in balancing computational efficiency and predictive performance. The evaluation was carried out using key performance metrics such as accuracy, inference time, and model size, which are critical indicators for determining the suitability of models in real-time and resource-constrained environments. The results demonstrate that lightweight models significantly reduce computational load while still maintaining reasonable levels of accuracy suitable for practical deployment.

One of the most noticeable improvements observed during experimentation was the reduction in inference time. Compared to traditional deep convolutional neural network (CNN) models, lightweight architectures required fewer computations per prediction, resulting in faster processing speeds. This improvement is particularly important for applications such as autonomous driving and industrial automation, where decisions must be made within very short time intervals. Faster inference enables systems to respond more quickly to environmental changes, which improves overall system responsiveness and operational safety. The reduced latency also makes these models highly suitable for edge computing scenarios, where real-time processing is essential.

Another important observation relates to model size reduction. After applying optimization techniques such as pruning and quantization, the total number of parameters within the models decreased substantially. Pruning helped eliminate redundant or less significant connections in the network, while quantization reduced the numerical precision of model weights. These techniques collectively resulted in smaller model footprints, making them easier to store and deploy on devices with limited memory capacity. From a practical deployment perspective, this reduction in storage requirement is highly beneficial, particularly for embedded systems and IoT-based platforms that cannot support large-scale models.

In addition to size reduction, the computational complexity of the optimized models also decreased noticeably. This reduction translated into lower power consumption during inference, which is an important factor in battery-powered or energy-sensitive systems. In smart manufacturing environments, for example, energy-efficient models can reduce operational costs and improve system sustainability. Similarly, in autonomous driving applications, efficient models can help maintain consistent performance without excessive hardware demands.

Although lightweight models provided several advantages in terms of efficiency, a minor decrease in accuracy was observed when compared to larger, more complex neural network architectures. This accuracy drop is expected due to the removal of redundant parameters and reduction in precision during optimization. However, the decrease was not significant enough to compromise overall usability in most practical scenarios. In many real-world systems, achieving slightly lower accuracy is acceptable if it results in faster processing and lower computational requirements. Therefore, the tradeoff between accuracy and efficiency appears to be balanced effectively in lightweight models.

Another key observation was the stability of model performance across different types of input data. The models demonstrated consistent behavior when tested on diverse visual inputs, including images related to defect detection and general object recognition tasks. This consistency indicates that lightweight architectures can generalize reasonably well across varying environmental conditions, which is essential for reliable deployment in dynamic real-world settings.

Furthermore, the experimental results highlight the importance of selecting appropriate optimization strategies based on the application requirements. For example, aggressive pruning can significantly reduce model size but may lead to noticeable performance degradation if not applied carefully. Similarly, quantization techniques must be optimized to ensure that numerical precision loss does not adversely affect prediction accuracy. Therefore, achieving optimal performance requires careful tuning of optimization parameters to maintain a suitable balance between efficiency and predictive capability.

Overall, the findings from this study confirm that lightweight computer vision models provide a practical and scalable solution for deployment in environments with limited computational resources. By reducing inference time, model size, and computational complexity while maintaining accept-

able accuracy levels, these models demonstrate strong potential for real-world applications. The results support the idea that efficiency-focused model design is not only beneficial but also necessary for enabling widespread adoption of artificial intelligence technologies in smart manufacturing systems and autonomous driving platforms.

V. CONCLUSION

This study focused on improving the practicality and deployability of computer vision models for real-world applications such as smart manufacturing and autonomous driving. Throughout this work, the primary objective was to address the limitations of conventional deep learning models, which often require high computational resources and large memory capacity. While these traditional models achieve high accuracy, their heavy computational demands make them unsuitable for deployment on resource-constrained platforms such as embedded systems and edge devices. Therefore, this research emphasized the development of lightweight and efficient computer vision solutions that can operate effectively under real-world constraints.

To achieve this objective, model optimization techniques such as pruning and quantization were applied to reduce model complexity and improve computational efficiency. Pruning enabled the removal of redundant parameters and less significant connections within the neural network, thereby reducing the overall number of computations required during inference. Quantization further contributed to efficiency by lowering the precision of model weights and activations, resulting in reduced memory usage and faster execution speeds. The application of these techniques demonstrated that it is possible to significantly reduce model size and computational requirements without causing a substantial decline in predictive performance.

The results obtained from this study highlight the feasibility of deploying lightweight computer vision models in real-time applications. By achieving faster inference times and smaller model footprints, the optimized models become suitable for integration into edge devices, embedded systems, and Internet of Things (IoT) platforms. Such deployment capabilities are particularly beneficial in applications where immediate decision-making is required, such as industrial defect detection systems and autonomous navigation modules. The improved efficiency of lightweight models also supports reduced power consumption, making them more sustainable and cost-effective for long-term operation.

Another important outcome of this research is the demonstration of the tradeoff between accuracy and efficiency. While lightweight models may show slightly lower accuracy compared to large-scale deep learning models, the reduction in latency and resource consumption often outweighs this limitation in practical scenarios. In many real-world applications, achieving reliable performance within limited computational constraints is more valuable than achieving maximum accuracy under ideal conditions. Therefore, lightweight model

design plays a crucial role in enabling scalable and practical artificial intelligence solutions.

Looking ahead, several opportunities exist for extending this work and improving the robustness of lightweight computer vision systems. Future research may involve testing these optimized models directly in real-world deployment environments to better evaluate their performance under varying conditions such as lighting changes, noise, and dynamic environments. Additionally, integrating multimodal data sources—such as combining visual data with sensor or textual information—could enhance the accuracy and adaptability of the system. Exploring advanced techniques such as federated learning, edge AI frameworks, and hardware-aware model optimization may further improve system performance and scalability. These approaches can also enhance privacy preservation and enable distributed learning across multiple devices without requiring centralized data storage.

In conclusion, this research contributes toward the development of efficient and deployable computer vision models that balance accuracy, speed, and computational requirements. By focusing on lightweight architectures and optimization techniques, this study demonstrates a practical pathway for bridging the gap between theoretical deep learning advancements and their real-world implementation. The findings emphasize the importance of designing resource-efficient models that are not only accurate but also capable of operating reliably in constrained environments. Such advancements are essential for accelerating the adoption of intelligent systems in modern industrial automation and autonomous technologies.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," Proc. NIPS, 2012.
- [2] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," Proc. ICLR, 2021.
- [3] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.
- [4] M. Tan and Q. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," Proc. ICML, 2019.
- [5] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning," Proc. ICLR, 2016.
- [6] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," Nature, 2015.
- [7] J. Redmon et al., "You only look once: Unified, real-time object detection," Proc. CVPR, 2016.
- [8] B. Wu et al., "FBNet: Hardware-aware efficient convnet design," Proc. CVPR, 2019.
- [9] N. Lane et al., "DeepX: A software accelerator for low-power deep learning inference," Proc. IPSN, 2016.
- [10] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino*, O'Reilly Media, 2019.